Databricks Data Engineering

Beginner-to-Mock-Client Interview Guide

Beginner memory: Learn every topic as one connected retail pipeline: Source -> Bronze -> Silver -> Gold -> BI.

Prepared for Soumiya Ayyappan | Beginner-friendly explanations, practical analogies, syntax, interview answers, MCQs and scenarios.

Your learning order	Topics
1. Platform basics	Databricks, Lakehouse, Spark, compute, notebooks
2. Pipeline core	ETL/ELT, medallion, Auto Loader, batch/streaming
3. Incremental processing	CDC, MERGE, checkpointing
4. Orchestration	Jobs, workflows, tasks, dependencies, retries
5. Scale and performance	Partitions, shuffle, joins, skew, EXPLAIN, Spark UI
6. Delta and governance	Delta Lake, OPTIMIZE, clustering, VACUUM, Unity Catalog
7. Interview readiness	SQL, Python, Scala, Hadoop, Kafka, scenarios

1. The Big Picture: What Databricks Is

Databricks is a cloud data and AI platform used to ingest, process, store, govern and analyze large amounts of data. A simple analogy is a modern data factory.

Factory item	Databricks meaning
Warehouse	Cloud storage / data lake
Workers	Spark executors
Team leader	Spark driver
Work desk	Notebook
Reliable storage system	Delta Lake
Security and inventory office	Unity Catalog
Automatic shift schedule	Databricks Workflows / Jobs
Business counter	Databricks SQL / Power BI

Interview answer: Databricks is a lakehouse platform. In data engineering, I can use Spark to process data, Delta Lake to store reliable tables, Workflows to orchestrate pipelines and Unity Catalog to govern access and lineage.

Lakehouse

A lakehouse combines the flexibility and lower-cost storage style of a data lake with table reliability, governance and analytical capabilities associated with a warehouse.

Beginner memory: Data lake = flexible storage. Data warehouse = structured analytics. Lakehouse = combines useful qualities of both.

Compute, cluster and SQL warehouse

Compute provides CPU and memory to execute work. For Spark data engineering, compute runs the driver and executors. A SQL warehouse is compute optimized for SQL and BI workloads.

Do not confuse: A Databricks SQL warehouse is compute. A data warehouse is a broader data architecture/storage concept.

2. Apache Spark Foundations

Driver, executors and cluster manager

Component	Layman analogy	Responsibility
Driver	Team leader	Plans and coordinates the Spark application
Executor	Worker	Runs tasks on data partitions
Cluster manager	Resource allocator	Allocates compute resources; examples include Standalone, YARN and Kubernetes

SparkSession

SparkSession is the main entry point for Spark structured APIs such as DataFrames and Spark SQL.

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
.appName("SalesPipeline")
.getOrCreate())
```

Interview answer: SparkSession is the entry point for interacting with Spark DataFrames and SQL. Java main() and Python __init__() are only analogies; they have different technical purposes.

RDD and DataFrame

Concept	Meaning	Use
RDD	Resilient Distributed Dataset	Lower-level distributed collection; strong control but fewer optimizations
DataFrame	Distributed table with named columns and schema	Preferred for most structured data engineering because Spark can optimize it

Beginner memory: RDD = distributed collection. DataFrame = distributed table.

Transformation, action and lazy evaluation

Type	Examples	Meaning
Transformation	select, filter, map, groupBy	Builds a new logical result; usually evaluated lazily
Action	show, count, collect, write	Requests a result and triggers execution

Lazy evaluation means Spark first builds an execution plan and performs the work when an action requires a result.

3. ETL, ELT and Medallion Architecture

ETL vs ELT

Pattern	Order	Best beginner interpretation
ETL	Extract -> Transform -> Load	Clean before loading into the final target
ELT	Extract -> Load -> Transform	Load raw data first, then transform using platform compute

Interview answer: In a Databricks lakehouse, an ELT-style approach is common: ingest raw data into Bronze, clean and validate it into Silver, then create business-ready Gold data.

Bronze, Silver and Gold

Layer	Contains	Typical work
Bronze	Raw/source-like data	Ingest, preserve source fields, add ingestion metadata
Silver	Cleaned and validated data	Types, null handling, deduplication, standardization, joins
Gold	Business-ready data	KPIs, aggregates, reporting tables

```
Sources -> Auto Loader -> Bronze -> Clean/Validate -> Silver -> Aggregate -> Gold -> Power BI
```

Data-quality checks

Common checks include required-column validation, schema validation, null checks, duplicate checks, datatype checks, date/range checks and business-rule validation.

Interview answer: I would not silently delete bad records. I would define quality rules, process valid records, quarantine invalid records with rejection reasons and make failures observable through monitoring.

4. Auto Loader and Ingestion

Databricks Auto Loader incrementally processes newly arriving files in cloud object storage. The key source format name to recognize is cloudFiles.

```
orders = (spark.readStream
.format("cloudFiles")
.option("cloudFiles.format", "json")
.option("cloudFiles.schemaLocation", "/checkpoints/orders_schema")
.load("/landing/orders"))
```

Beginner memory: Auto Loader asks: Which new files arrived? It is commonly used for Source -> Bronze ingestion.

Auto Loader vs autoscaling

Term	Purpose
Auto Loader	Incremental file ingestion
Autoscaling	Adjusting compute capacity according to workload

Do not confuse: Auto Loader is not autoscaling.

Schema inference, evolution and rescued data

Schema inference detects column types. Schema evolution handles approved incoming schema changes. A rescued-data mechanism can preserve unexpected data for investigation instead of silently dropping it.

Interview answer: For new cloud files, I would use Auto Loader to ingest incrementally into Bronze, maintain schema and checkpoint locations separately, and monitor invalid or unexpected records.

5. Full Load, Incremental Load, CDC and MERGE

Full vs incremental

Approach	Meaning	Trade-off
Full load	Reload all required records	Simple, but expensive for large history
Incremental load	Process new or changed records	Efficient, but needs reliable change tracking and recovery

CDC

Change Data Capture identifies inserts, updates and deletes in source data. CDC answers: Which records changed?

MERGE / UPSERT

MERGE applies source changes to a target. UPSERT usually means update an existing key and insert a new key.

```
MERGE INTO customers AS t
USING customer_changes AS s
ON t.customer_id = s.customer_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Beginner memory: Auto Loader = new files. CDC = changed records. MERGE = apply changes.

Interview answer: If 50,000 of 100 million rows change daily, I would consider incremental CDC processing and Delta MERGE instead of automatically reloading the entire history.

Idempotency

An idempotent pipeline can safely rerun the same input without creating incorrect duplicates or repeated business effects. Stable keys, checkpoints and deterministic MERGE rules help.

6. Batch, Streaming, Windows and Checkpointing

Batch vs streaming

Mode	Example	Choose when
Batch	Nightly sales report	Periodic processing meets the latency requirement
Streaming	Fraud events or live clickstream	Continuously arriving data needs low-latency processing

Streaming time window

A time window groups events into intervals, such as calculating transaction count for every five-minute period.

```
from pyspark.sql.functions import window

result = events.groupBy(
window("event_time", "5 minutes")
).count()
```

Watermark

A watermark defines how long the streaming query should wait for late events while controlling retained state. It is a trade-off: a longer delay accepts later data but retains more state.

Checkpointing

Checkpointing stores streaming progress and state information used for fault recovery. It is like a bookmark, not a copy of the entire source dataset.

> **Interview answer:** For streaming, I would configure a durable and unique checkpoint location, use event-time windows and watermarks when late data matters, and define how duplicates and reprocessing are handled.

7. Databricks Workflows and Pipeline Operations

Jobs, tasks and dependencies

Term	Meaning
Job / Workflow	The complete orchestrated process
Task	One unit of work, such as Bronze-to-Silver
Upstream	Provides input or runs before the current task
Downstream	Consumes output or depends on the current task
Dependency	Rule that controls execution order

```
Task A: Auto Loader -> Bronze
|
Task B: Bronze -> Silver
|
Task C: Silver -> Gold
```

Scheduling, parameters, retries and alerts

Scheduling starts the job at a required time. Parameters allow reusable notebooks to process dates, environments or tables. Retries handle temporary failures. Alerts and monitoring notify owners about failure, duration or data-quality issues.

Interview answer: I would configure dependencies so Gold runs only after successful Silver processing, use limited retries for transient failures, pass a processing-date parameter, and alert on final failure.

Failure-handling principle

Do not retry indefinitely. A network timeout may be transient, but corrupt input or an incompatible schema requires investigation or quarantine.

8. Scaling, Partitions, Shuffle and Skew

Horizontal, vertical and autoscaling

Concept	Meaning	Restaurant analogy
Horizontal scaling	Add more workers/machines	Hire more cooks
Vertical scaling	Add CPU/RAM to existing machines	Give a cook a larger, faster workstation
Autoscaling	Automatically adjust compute capacity	Schedule more cooks during rush hour

Partition

A Spark partition is a logical chunk of distributed data. Tasks process partitions in parallel. Too few partitions can limit parallelism; too many tiny partitions create scheduling and file overhead.

Shuffle

Shuffle is redistribution of data across partitions, commonly required by joins, groupBy, distinct, repartition and global sorting. It is expensive because it can involve network, serialization, memory and disk I/O.

Data skew

Skew occurs when a small number of keys contain much more data than other keys. One or a few tasks become stragglers while the rest finish quickly.

Interview answer: For skew, I would first confirm the uneven key distribution in Spark UI, then consider techniques such as salting, pre-aggregation, adaptive query execution, broadcast of a suitable small side or a better partitioning strategy.

9. Spark Join Strategies

Broadcast Hash Join

When one side is sufficiently small, Spark can distribute it to executors. Each executor joins its large-side partition locally, avoiding a shuffle of the large side.

```
from pyspark.sql.functions import broadcast
result = orders.join(broadcast(countries), "country_id")
```

Shuffle Hash Join

Both sides are shuffled by the join key; within matching partitions, Spark builds a hash structure for an appropriate side and probes it with the other side.

Sort-Merge Join

For suitable large equi-joins, Spark commonly shuffles both sides by the join key, sorts within partitions and merges matching keys.

Scenario	Likely idea	Important caution
1 TB fact + 5 MB dimension	Broadcast Hash Join	Only if the small side safely fits broadcast constraints
Two large equi-join tables	Sort-Merge Join is common	Optimizer decides using statistics, config and conditions
Uneven popular key	Potential skew	Inspect task durations and partition sizes

Interview answer: I do not choose only from table labels such as small or large. I inspect statistics, physical plan, join condition, memory safety, skew and Spark configuration.

10. Query Performance: EXPLAIN and Spark UI

EXPLAIN

EXPLAIN displays the query plan. In Spark, df.explain() helps inspect the logical and physical plans and the chosen join operators.

```
result.explain("formatted")

-- SQL
EXPLAIN FORMATTED
SELECT * FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
```

What to look for

Signal	Why it matters
BroadcastHashJoin / SortMergeJoin	Shows chosen physical join strategy
Exchange	Often indicates repartition/shuffle
Filter placement	Early filtering can reduce processed data
File scan and partition filters	Shows pruning and files/partitions read
Unexpected full scan	May indicate missing predicates or ineffective pruning

Spark UI

Spark UI shows jobs, stages, tasks, SQL plans, shuffle read/write, input/output, task duration, failures and executor behavior.

Interview answer: If a job is slow, I inspect the physical plan and Spark UI, identify the slow stage, check shuffle read/write, skewed task duration, spills, joins, scans and file counts, optimize the query, then resize compute only if needed.

Jobs, stages and tasks

An action creates a Spark job. Shuffle boundaries commonly divide work into stages. A stage contains tasks, often one task per partition.

11. Performance Techniques

Predicate pushdown and partition pruning

Predicate pushdown moves filters closer to the data source so less data is read. Partition pruning skips irrelevant physical partitions when filters use partition columns.

Caching

Cache data only when an expensive intermediate result is reused. Caching one-time data wastes memory and can make performance worse.

Small-file problem

Thousands of tiny files create metadata and task overhead. Compaction reduces file count and produces more efficient file sizes.

OPTIMIZE

OPTIMIZE compacts small Delta files. It improves data layout; it does not replace good ingestion and partitioning design.

Z-Ordering and Liquid Clustering

Z-Ordering colocates related values to improve data skipping for selected columns. Liquid clustering provides a flexible clustering approach and is generally preferred for many new Databricks table designs where supported. Do not combine every layout technique blindly.

VACUUM

VACUUM removes obsolete unreferenced data files older than the configured retention threshold. Aggressive retention can remove files needed for time travel or long-running readers.

> **Do not confuse:** OPTIMIZE compacts. Clustering/Z-Ordering improves layout for skipping. VACUUM removes old unreferenced files.

12. Delta Lake Essentials

Delta table and transaction log

A Delta table stores data files plus a transaction log that records table changes and versions. The log enables reliable table operations and historical versions.

ACID

Letter	Meaning	Simple idea
A	Atomicity	All parts of a transaction succeed or none do
C	Consistency	Valid rules remain satisfied
I	Isolation	Concurrent work does not expose incomplete changes
D	Durability	Committed changes persist

Schema enforcement vs evolution

Concept	Purpose
Schema enforcement	Reject incompatible or invalid writes
Schema evolution	Allow approved schema changes under controlled settings

Time travel

Time travel allows querying a retained previous table version for auditing, debugging or recovery. Availability depends on retained history and file retention.

```
SELECT * FROM sales VERSION AS OF 5;
```

13. Governance, Compliance and Security

Governance vs compliance

Governance defines how data is owned, described, protected, discovered and used. Compliance demonstrates adherence to applicable laws, regulations, contracts and organizational policies.

Unity Catalog

Unity Catalog provides centralized governance for Databricks data and AI assets, including object hierarchy, permissions, discovery, lineage and auditing capabilities.

```
Metastore -> Catalog -> Schema -> Table / View / Volume
```

Apache Atlas and Collibra

Tool	Beginner meaning
Apache Atlas	Open metadata and governance framework often associated with Hadoop ecosystems
Collibra	Enterprise data intelligence and governance platform
Unity Catalog	Databricks-native centralized governance layer

Security practices

Use least privilege, role-based access, service identities, secret management, encryption, auditing, sensitive-data classification, masking where required and separate development/test/production access.

Interview answer: Governance covers ownership, metadata, lineage, quality and access. Compliance ensures applicable requirements are followed and evidenced through controls and audit records.

14. Hadoop and Kafka Quick Revision

Hadoop

Component	Purpose
HDFS	Distributed storage
NameNode	Filesystem metadata
DataNode	Actual HDFS blocks
MapReduce	Distributed batch-processing model
YARN	Cluster resource management
Hive	SQL-style data warehouse/query layer in the Hadoop ecosystem

Kafka

Term	Meaning
Producer	Publishes records
Topic	Logical event category/stream
Partition	Ordered shard of a topic used for scale and parallelism
Broker	Kafka server storing partition data
Consumer	Reads records
Consumer group	Consumers cooperate to divide partitions

Beginner memory: Producer -> Topic partitions on Brokers -> Consumer group.

15. Scala, Python and Java Interview Essentials

Scala

Term	Correct meaning
val	Cannot be reassigned
var	Can be reassigned while remaining type-compatible; var does not mean static
lazy val	Initialized on first access, then retained
Any	Root/top type in Scala type hierarchy
if expression	Returns a value; Scala infers a common branch type

Python

Python is dynamically typed. Common interview areas include list/dictionary operations, functions, classes, exceptions, shallow/deep copy and immutability.

```
import copy
a = [[1, 2], [3, 4]]
b = copy.copy(a) # nested objects shared
c = copy.deepcopy(a) # nested objects copied recursively
```

Java

Concept	Meaning
Static typing	Variable types are checked at compile time
this	Reference to current object
Overloading	Same method name, different parameter lists
Heap	Objects created with new generally live here
Stack	Method frames and local variables/references
Autoboxing	Primitive to wrapper conversion, e.g. int to Integer
String	Immutable

16. SQL and PySpark Syntax for Mock Client

SQL essentials

```
SELECT department, AVG(salary) AS avg_salary
FROM employee
WHERE active = true
GROUP BY department
HAVING AVG(salary) > 50000
ORDER BY avg_salary DESC;
```

Join

```
SELECT e.name, d.department_name
FROM employee e
LEFT JOIN department d
ON e.dept_id = d.dept_id;
```

Window function

```
SELECT name, department, salary,
DENSE_RANK() OVER (
PARTITION BY department
ORDER BY salary DESC
) AS salary_rank
FROM employee;
```

PySpark

```
clean = (orders
.filter("amount > 0")
.dropDuplicates(["order_id"])
.withColumnRenamed("amt", "amount"))

summary = clean.groupBy("region").sum("amount")
```

Latest record per key

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, col

w = Window.partitionBy("customer_id").orderBy(col("updated_at").desc())
latest = df.withColumn("rn", row_number().over(w)).filter("rn = 1")
```

17. How to Answer: A Complete Databricks Project

Scenario: A retail client receives thousands of JSON order files daily. Raw data must be retained, bad records handled, customer updates applied incrementally, and management needs daily revenue dashboards.

Interview answer: I would use Auto Loader to incrementally ingest cloud files into a Bronze Delta table with schema and checkpoint locations. I would transform Bronze to Silver using schema validation, datatype correction, null handling, deterministic deduplication and quarantine for invalid records. If customer records arrive as changes, I would use CDC logic and Delta MERGE to apply inserts and updates idempotently. From Silver, I would create Gold aggregates such as daily revenue by product and region. I would orchestrate the dependent tasks through Databricks Workflows with parameters, retries, monitoring and alerts. Unity Catalog would govern permissions and lineage. For performance, I would inspect EXPLAIN and Spark UI, then address scans, shuffles, skew, file sizes and join strategy before resizing compute.

Why this answer is strong

It connects ingestion, storage, transformation, incremental processing, orchestration, governance and performance instead of listing disconnected definitions.

18. Performance Troubleshooting Framework

When asked, "The pipeline is slow. What will you do?" use this order:

Step	Action
1. Define the problem	Which job, stage, query, input size and expected SLA?
2. Inspect plan	Use EXPLAIN FORMATTED / df.explain("formatted")
3. Inspect Spark UI	Find slow stage, long tasks, shuffle, spills and failures
4. Check data access	File count, full scans, partition pruning and pushed filters
5. Check joins	Broadcast safety, sort-merge behavior, statistics and skew
6. Check partitioning	Too few partitions, too many tiny tasks or uneven keys
7. Check Delta layout	Small files, compaction and appropriate clustering
8. Check code	Unnecessary collect, repeated actions, Python UDFs and recomputation
9. Resize last	Scale compute after fixing avoidable inefficiencies

Interview answer: I optimize the workload before blindly increasing cluster size, because more compute does not automatically fix skew, a bad join, a full scan or millions of tiny files.

19. Scenario-Based Interview Questions

1. New JSON files arrive every hour. Which feature?

Auto Loader, usually landing into Bronze.

2. Auto Loader or autoscaling?

Auto Loader ingests files; autoscaling adjusts compute.

3. Only 0.1% of a table changes daily. Full or incremental?

Consider incremental processing and CDC.

4. Existing key update, new key insert. Which operation?

Delta MERGE / UPSERT.

5. Silver fails. Should dependent Gold run?

No, not if successful Silver output is required.

6. A 1 TB fact table joins a 5 MB dimension. First strategy to evaluate?

Broadcast Hash Join, after checking safety and plan.

7. Two 500 GB tables join on an equality key. Common strategy?

Sort-Merge Join is common, subject to the optimizer.

8. One task runs 20 minutes and others run 30 seconds. Suspect?

Data skew or an unusually large partition.

9. How do you see the chosen join strategy?

EXPLAIN / df.explain and Spark UI SQL plan.

10. Thousands of tiny Delta files. What helps?

Compaction with OPTIMIZE, plus fixing upstream write patterns.

11. Need to delete obsolete Delta files. What command?

VACUUM, with careful retention considerations.

12. Need old table version. What feature?

Delta time travel, if retained.

13. Add more worker machines. Scaling type?

Horizontal scaling.

14. Increase RAM on each worker. Scaling type?

Vertical scaling.

15. Need transaction count every five minutes. What concept?

Streaming time window.

16. Why checkpoint?

Streaming progress/state and recovery.

17. Where should monthly revenue by region live?

Gold layer.

18. Where do you correct types and duplicates?

Typically Bronze-to-Silver.

19. What is lineage?

The path and transformations from source to consumers.

20. How do you protect sensitive tables?

Unity Catalog permissions, least privilege, audit and required masking/policies.

20. MCQ Final Test

1. Which format indicates Auto Loader?

A. deltaFiles B. cloudFiles C. autoLoad D. fileStream

Correct answer: B

2. Which layer preserves raw/source-like data?

A. Bronze B. Silver C. Gold D. SQL Warehouse

Correct answer: A

3. Which operation applies matched updates and unmatched inserts?

A. VACUUM B. OPTIMIZE C. MERGE D. CACHE

Correct answer: C

4. Adding workers is what type of scaling?

A. Vertical B. Horizontal C. Schema D. Static

Correct answer: B

5. What redistributes data across Spark partitions?

A. Shuffle B. Vacuum C. Catalog D. Checkpoint

Correct answer: A

6. Which plan operator may appear for a small dimension join?

A. BroadcastHashJoin B. NameNode C. Auto Loader D. Watermark

Correct answer: A

7. Which tool shows stage and task metrics?

A. Collibra B. Spark UI C. HDFS D. Git

Correct answer: B

8. What removes obsolete Delta files?

A. OPTIMIZE B. VACUUM C. ZORDER D. MERGE

Correct answer: B

9. Which layer contains business KPIs?

A. Bronze B. Silver C. Gold D. Landing

Correct answer: C

10. Which describes CDC?

A. Add workers B. Detect record changes C. Compact files D. Rank rows

Correct answer: B

11. Which is the root Scala type?

A. String B. Any C. var D. lazy

Correct answer: B

12. Where are HDFS actual blocks stored?

A. NameNode B. DataNode C. YARN D. Hive

Correct answer: B

13. Which Kafka component publishes records?

A. Consumer B. Broker C. Producer D. Partition

Correct answer: C

14. Python variable types are mainly determined when?

A. Runtime B. Never C. SQL compile D. HDFS load

Correct answer: A

15. What does df.explain() show?

A. User permissions B. Query plan C. File contents D. Broker list

Correct answer: B

21. Rapid Revision Sheet

Term	One-line memory
Databricks	Lakehouse platform for data engineering, analytics and AI
SparkSession	Entry point for DataFrame and SQL APIs
RDD	Resilient distributed collection
DataFrame	Distributed table with schema
ETL	Transform before final load
ELT	Load raw first, transform later
Bronze / Silver / Gold	Raw / Clean / Business-ready
Auto Loader	Incremental cloud-file ingestion
CDC	Detect inserted, updated and deleted records
MERGE	Apply updates/inserts to target
Checkpoint	Streaming progress and recovery state
Workflow	Orchestrates scheduled dependent tasks
Partition	Logical chunk processed in parallel
Shuffle	Redistribution across partitions
Skew	Uneven data creates slow tasks
Broadcast join	Send suitable small side to executors
Sort-merge join	Shuffle, sort and merge large equi-join sides
EXPLAIN	Inspect query plan
Spark UI	Inspect jobs, stages, tasks and metrics
OPTIMIZE	Compact Delta files
Clustering / ZORDER	Improve layout and data skipping
VACUUM	Remove obsolete files after retention
Unity Catalog	Central governance, permissions and lineage
Horizontal / Vertical	More machines / bigger machines

22. Final 60-Second Mock-Client Answer

Interview answer: I work in the Databricks data-engineering domain. I would ingest newly arriving cloud files using Auto Loader into Bronze Delta tables, preserving source data and ingestion metadata. I would transform the data into Silver by validating schema, correcting types, handling nulls, applying business rules and resolving duplicates deterministically. I would create Gold tables containing business KPIs for Databricks SQL or Power BI. For changed records, I would use CDC logic and Delta MERGE for idempotent incremental processing. Databricks Workflows would schedule and orchestrate tasks with dependencies, retries and monitoring, while Unity Catalog would manage governance, access and lineage. If performance is slow, I would inspect EXPLAIN and Spark UI for scans, shuffle, skew, spills, file sizes and join strategy before changing compute.

Study method: read one chapter, explain it aloud without looking, write one syntax example, then answer the scenarios. Repeat difficult topics until the explanation becomes natural.